

| 사물인터넷 실습 |

Internet of Things and Laboratory

TinyML
(2)

| TinyML 환경 구축 (1/3)

아두이노의 성능 제약

- SRAM 2KB
 - 메모리 계층 구조 활용 필요
- non-FPU
 - 부동 소수점 연산 대신 정수 연산 필요
- Flash 32KB
 - 모델 크기 최적화
- 16MHz Single core
 - 연산량이 적은 모델 사용

| TinyML 환경 구축 (2/3)

학습 환경 구축 (데스크탑)

- 파이썬 가상환경 생성

```
# 가상환경 생성
python -m venv tinym1_env
# 활성화 - Windows
tinym1_env\Scripts\activate
# 활성화 - macOS / Linux
source tinym1_env/bin/activate
# 비활성화
deactivate
```

- 라이브러리 설치

```
pip install pyserial
pip install numpy
pip install pandas
pip install scikit-learn
pip install matplotlib
```

| TinyML 환경 구축 (3/3)

아두이노 데이터를 CSV에 저장

아두이노 랜덤 데이터 예제 코드

```
void setup() {  
    Serial.begin(9600);  
    randomSeed(analogRead(0));  
}  
void loop() {  
    float a = random(0, 1001) / 1000.0;  
    float b = random(0, 1001) / 1000.0;  
    float c = random(0, 1001) / 1000.0;  
  
    Serial.print(a, 3);  
    Serial.print(",");  
    Serial.print(b, 3);  
    Serial.print(",");  
    Serial.println(c, 3); // 마지막은 println으로 줄바꿈  
  
    delay(500); // 0.5초마다 전송  
}
```

```

import serial      #데스크탑에서 실행 #ArduinoIDE 시리얼창 끄고 실행(Permission Error)
import csv
from datetime import datetime
ser = serial.Serial('COM##', 9600, timeout=1) #포트는 본인의 아두이노에 맞게 변경
filename = f"data_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
with open(filename, 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['a', 'b', 'c']) # 헤더
    print(f"저장 중: {filename}")
    print("종료: Ctrl+C")
    try:
        ser.readline()
        while True:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if not line:
                continue
            parts = line.split(',')
            if len(parts) != 3:
                continue
            try:
                a, b, c = float(parts[0]), float(parts[1]), float(parts[2])
            except ValueError:
                continue
            writer.writerow([a, b, c])
            f.flush()
            print(f"a={a:.3f} b={b:.3f} c={c:.3f}")
    except KeyboardInterrupt:
        print("종료")

finally:
    ser.close()

```

```

1  time,light,temp,pot
2  13:08:48,0.380,0.342,0.315
3  13:08:48,0.298,0.293,0.291
4  13:08:48,0.281,0.280,0.278
5  13:08:49,0.274,0.274,0.271
6  13:08:49,0.268,0.269,0.265
7  13:08:50,0.273,0.275,0.270
8  13:08:50,0.275,0.277,0.274
9  13:08:51,0.274,0.276,0.272
10 13:08:51,0.273,0.275,0.271
11 13:08:52,0.262,0.263,0.260
12 13:08:52,0.253,0.252,0.250
13 13:08:53,0.256,0.258,0.255
14 13:08:53,0.263,0.266,0.261
15 13:08:54,0.266,0.268,0.263
16 13:08:54,0.258,0.259,0.256
17 13:08:55,0.251,0.251,0.249
18 13:08:55,0.257,0.258,0.255
19 13:08:56,0.263,0.266,0.261
20 13:08:56,0.262,0.263,0.259

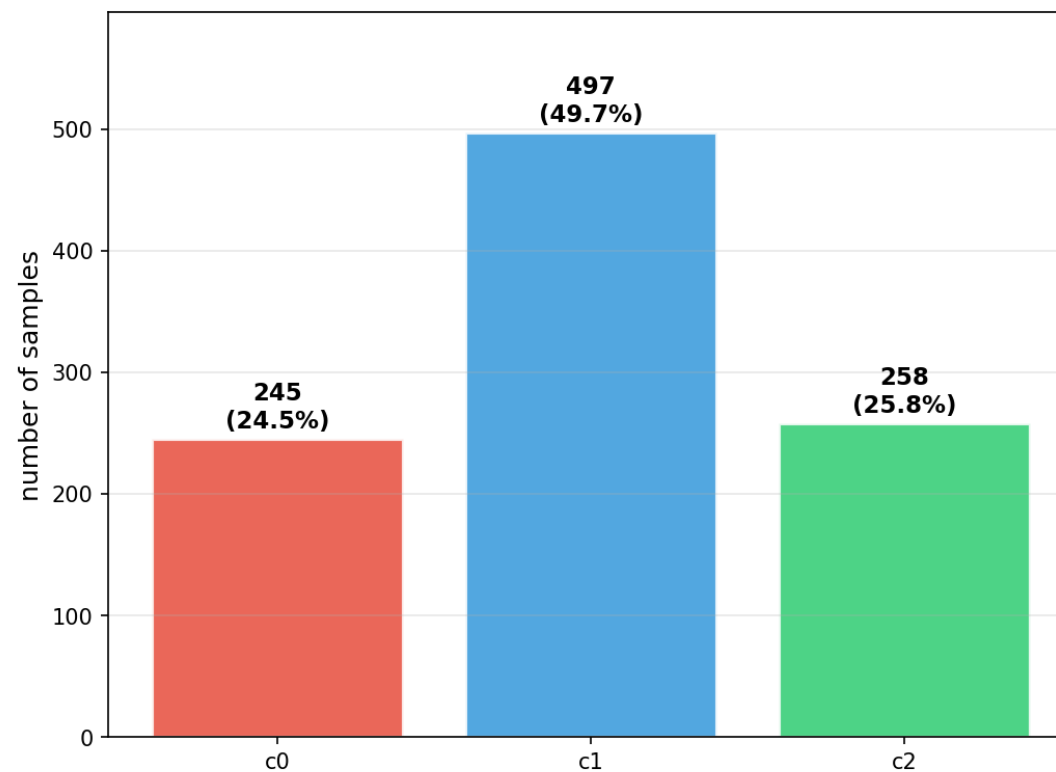
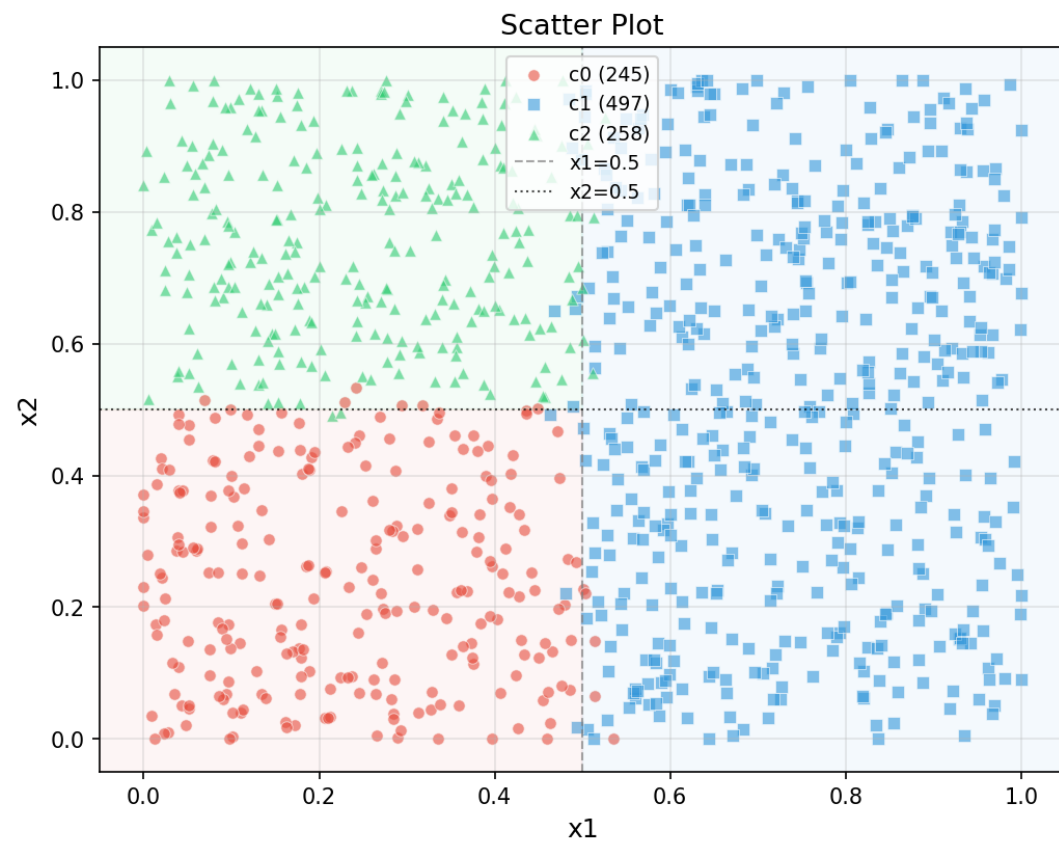
```

데이터 시각화

데이터 시각화

- 예제 코드 참고

Dataset Visualization



| 모델 학습(1/6)

하이퍼 파라미터 및 상수

```
CSV_FILE = 'dataset.csv'
LR        = 0.05
EPOCHS    = 2000
BATCH     = 16
INPUT_SIZE = 2          # 입력층 크기 (x1, x2)
OUTPUT_SIZE = 3         # 출력층 크기 (c0, c1, c2)
HIDDEN_LAYERS = [8]     # 은닉층 구조 - 원하는 대로 변경

# 예시:
# HIDDEN_LAYERS = [8]           → 2 - 8 - 3
# HIDDEN_LAYERS = [16]          → 2 - 16 - 3
# HIDDEN_LAYERS = [16, 8]       → 2 - 16 - 8 - 3
# HIDDEN_LAYERS = [32, 16, 8]   → 2 - 32 - 16 - 8 - 3
# HIDDEN_LAYERS = [64, 32]      → 2 - 64 - 32 - 3
```

| 모델 학습(2/6)

데이터 로드

```
df = pd.read_csv(CSV_FILE)
X = df[['x1', 'x2']].values.astype(float)
Y = df[['c0', 'c1', 'c2']].values.astype(float)

print(f"\n총 샘플: {len(df)}개")
y_cls = np.argmax(Y, axis=1)
for i, name in enumerate(['c0', 'c1', 'c2']):
    cnt = np.sum(y_cls == i)
    print(f" {name}: {cnt}개 ({cnt/len(df)*100:.1f}%)")

X_train, X_val, Y_train, Y_val = train_test_split(
    X, Y, test_size=0.2, random_state=42,
    stratify=np.argmax(Y, axis=1)
)
print(f"\n학습: {len(X_train)}개   검증: {len(X_val)}개\n")
```

=====

총 샘플: 1000개

c0: 245개 (24.5%)

c1: 497개 (49.7%)

c2: 258개 (25.8%)

학습: 800개 검증: 200개

=====

| 모델 학습(3/6)

모델 구조 정의

```
LAYER_SIZES = [INPUT_SIZE] + HIDDEN_LAYERS + [OUTPUT_SIZE]
N_LAYERS     = len(LAYER_SIZES)

print("=" * 55)
print("모델 구조:")
structure = ' → '.join(
    f"{s}({'ReLU' if i < N_LAYERS-2 else 'Softmax'})"
    if i > 0 else str(s)
    for i, s in enumerate(LAYER_SIZES)
)
print(f" {structure}")

total_params = sum(
    LAYER_SIZES[i] * LAYER_SIZES[i+1] + LAYER_SIZES[i+1]
    for i in range(N_LAYERS - 1)
)
print(f"총 파라미터 수: {total_params}개")
print(f"Flash 사용량: {total_params * 4}바이트 (float32 기준)")
print("=" * 55)
```

```
=====
모델 구조:
  2(입력) → 8(ReLU) → 3(Softmax)
총 파라미터 수: 51개
Flash 사용량: 204바이트 (float32 기준)
=====
```

| 모델 학습(4/6)

활성화 함수 및 손실 함수

```
def relu(z):  
    return np.maximum(0, z)  
  
def relu_grad(z):  
    return (z > 0).astype(float)  
  
def softmax(z):  
    z = z - np.max(z, axis=1, keepdims=True)  
    e = np.exp(z)  
    return e / np.sum(e, axis=1, keepdims=True)  
  
def cross_entropy(y_pred, y_true):  
    y_pred = np.clip(y_pred, 1e-9, 1.0)  
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
```

| 모델 학습(5/6)

순전파와 역전파

```
def forward(X, weights, biases):
    activations = [X]    # 각 레이어 출력 저장
    z_values     = []     # 선형 출력 저장

    current = X
    for i, (W, b) in enumerate(zip(weights, biases)):
        z = current @ W + b
        z_values.append(z)

        # 마지막 레이어 → Softmax, 나머지 → ReLU
        if i == len(weights) - 1:
            current = softmax(z)
        else:
            current = relu(z)

        activations.append(current)

    return z_values, activations
```

```
def backward(Y, z_values, activations, weights):
    N      = activations[0].shape[0]
    dW_list = [None] * len(weights)
    db_list = [None] * len(weights)

    # 출력층 델타 (Softmax + Cross Entropy)
    delta = (activations[-1] - Y) / N

    for i in reversed(range(len(weights))):
        dW_list[i] = activations[i].T @ delta
        db_list[i] = np.sum(delta, axis=0)

        if i > 0:
            delta = (delta @ weights[i].T) *
relu_grad(z_values[i - 1])

    return dW_list, db_list
```

모델 학습(6/6)

모델 학습

```
for epoch in range(1, EPOCHS + 1):
    idx = np.random.permutation(len(X_train))
    X_shuf = X_train[idx]
    Y_shuf = Y_train[idx]

    for s in range(0, len(X_train), BATCH):
        Xb = X_shuf[s:s+BATCH]
        Yb = Y_shuf[s:s+BATCH]

        z_vals, acts = forward(Xb, weights, biases)
        dW_list, db_list = backward(Yb, z_vals, acts, weights)

        for i in range(len(weights)):
            weights[i] -= LR * dW_list[i]
            biases[i] -= LR * db_list[i]
```

=====

학습 시작

=====

Epoch	100	Train Loss:0.1182	Acc:0.954	Val Loss:0.0947	Acc:0.970
Epoch	200	Train Loss:0.0934	Acc:0.954	Val Loss:0.0646	Acc:0.975
Epoch	300	Train Loss:0.0852	Acc:0.963	Val Loss:0.0549	Acc:0.985
Epoch	400	Train Loss:0.0783	Acc:0.969	Val Loss:0.0461	Acc:0.980
Epoch	500	Train Loss:0.0743	Acc:0.966	Val Loss:0.0452	Acc:0.980
Epoch	600	Train Loss:0.0716	Acc:0.968	Val Loss:0.0441	Acc:0.985
Epoch	700	Train Loss:0.0694	Acc:0.973	Val Loss:0.0415	Acc:0.985
Epoch	800	Train Loss:0.0689	Acc:0.969	Val Loss:0.0403	Acc:0.980
Epoch	900	Train Loss:0.0685	Acc:0.965	Val Loss:0.0426	Acc:0.985
Epoch	1000	Train Loss:0.0665	Acc:0.973	Val Loss:0.0399	Acc:0.985
Epoch	1100	Train Loss:0.0659	Acc:0.970	Val Loss:0.0416	Acc:0.990
Epoch	1200	Train Loss:0.0673	Acc:0.969	Val Loss:0.0442	Acc:0.985
Epoch	1300	Train Loss:0.0648	Acc:0.973	Val Loss:0.0404	Acc:0.980
Epoch	1400	Train Loss:0.0663	Acc:0.969	Val Loss:0.0448	Acc:0.990
Epoch	1500	Train Loss:0.0684	Acc:0.970	Val Loss:0.0403	Acc:0.980
Epoch	1600	Train Loss:0.0643	Acc:0.975	Val Loss:0.0404	Acc:0.980
Epoch	1700	Train Loss:0.0636	Acc:0.971	Val Loss:0.0407	Acc:0.980
Epoch	1800	Train Loss:0.0637	Acc:0.973	Val Loss:0.0403	Acc:0.980
Epoch	1900	Train Loss:0.0654	Acc:0.975	Val Loss:0.0414	Acc:0.985
Epoch	2000	Train Loss:0.0641	Acc:0.973	Val Loss:0.0446	Acc:0.985

| 모델 학습 완료

모델 학습 결과 예시 (weight.h)

```
const float W1[2][8] PROGMEM = {
    {6.480684f, -0.431309f, 0.398140f, 4.919894f, 0.374030f, -0.234137f, 1.187580f, 2.298036f},
    {0.860488f, 7.003561f, -0.548935f, 0.134427f, 3.240387f, -1.913280f, -1.865195f, -1.727819f}
};

const float b1[8] PROGMEM = {-2.823417f, -3.139245f, -0.287482f, -2.163809f, -0.686982f, 0.000000f,
-0.333659f, -0.107104f};

const float W2[8][3] PROGMEM = {
    {-3.794620f, 4.662945f, -1.671629f},
    {-6.150858f, 3.349663f, 2.714979f},
    {-0.003793f, -0.616072f, -0.330936f},
    {-2.550099f, 3.492330f, -1.274418f},
    {-1.677169f, -1.332393f, 2.262542f},
    {0.926139f, -0.006749f, -0.528855f},
    {-0.038336f, 0.173696f, -0.230078f},
    {-2.033847f, 1.621881f, -1.133531f}
};

const float b2[3] PROGMEM = {5.698262f, -4.253442f, -1.444820f};
```

| 실시간 추론 (1/3)

헤더 및 상수, softmax 함수 (weights.h는 .ino 파일과 동일한 디렉토리에 저장)

```
#include <math.h>
#include "weights.h"

#define IN_SIZE    2
#define HID_SIZE   8
#define OUT_SIZE   3

const char* CLASS_NAMES[OUT_SIZE] = {"c0", "c1", "c2"};

void softmax(float* z, float* out) {
    float mx = z[0];
    for (int i = 1; i < OUT_SIZE; i++)
        if (z[i] > mx) mx = z[i];

    float sum = 0;
    for (int i = 0; i < OUT_SIZE; i++) {
        out[i] = exp(z[i] - mx);
        sum += out[i];
    }
    for (int i = 0; i < OUT_SIZE; i++)
        out[i] /= sum;
}
```

| 실시간 추론 (2/3)

순전파

```
int predict(float x1, float x2, float* prob_out) {
    float input[IN_SIZE] = {x1, x2};
    float h[HID_SIZE];
    float z2[OUT_SIZE];
    float prob[OUT_SIZE];

    // 히든층 (ReLU)
    for (int j = 0; j < HID_SIZE; j++) {
        h[j] = pgm_read_float(&b1[j]);
        for (int i = 0; i < IN_SIZE; i++)
            h[j] += input[i] * pgm_read_float(&W1[i][j]);
        if (h[j] < 0) h[j] = 0;
    }

    // 출력층
    for (int k = 0; k < OUT_SIZE; k++) {
        z2[k] = pgm_read_float(&b2[k]);
        for (int j = 0; j < HID_SIZE; j++)
            z2[k] += h[j] * pgm_read_float(&W2[j][k]);
    }
    softmax(z2, prob);

    if (prob_out != nullptr)
        for (int k = 0; k < OUT_SIZE; k++)
            prob_out[k] = prob[k];

    int pred = 0;
    for (int k = 1; k < OUT_SIZE; k++)
        if (prob[k] > prob[pred]) pred = k;

    return pred;
}
```

| 실시간 추론 (3/3)

loop()

```
float x1 = randomFloat();
float x2 = randomFloat();

float prob[OUT_SIZE];
int cls = predict(x1, x2, prob);

Serial.print(F("x1=")); Serial.print(x1, 4);
Serial.print(F(" x2=")); Serial.print(x2, 4);
Serial.print(F(" → ")); Serial.print(CLASS_NAMES[cls]);
Serial.print(F(" ("));
for (int k = 0; k < OUT_SIZE; k++) {
    Serial.print(CLASS_NAMES[k]);
    Serial.print(F(":"));
    Serial.print(prob[k], 3);
    if (k < OUT_SIZE - 1) Serial.print(F(" "));
}
Serial.println(F(")"));

delay(500);
```

x1=0.5261	x2=0.0025	→	c1	(c0:0.159	c1:0.839	c2:0.002)
x1=0.2339	x2=0.0068	→	c0	(c0:0.999	c1:0.000	c2:0.001)
x1=0.6683	x2=0.7893	→	c1	(c0:0.000	c1:1.000	c2:0.000)
x1=0.6852	x2=0.3029	→	c1	(c0:0.000	c1:1.000	c2:0.000)
x1=0.3112	x2=0.1823	→	c0	(c0:0.999	c1:0.000	c2:0.001)
x1=0.1766	x2=0.3926	→	c0	(c0:0.990	c1:0.000	c2:0.010)
x1=0.3887	x2=0.9445	→	c2	(c0:0.000	c1:0.001	c2:0.999)
x1=0.2317	x2=0.8902	→	c2	(c0:0.000	c1:0.000	c2:1.000)
x1=0.8418	x2=0.7878	→	c1	(c0:0.000	c1:1.000	c2:0.000)

| 과제 11

데이터 수집 및 시각화

- 조이스틱과 스위치를 사용하여 데이터를 수집하고 이를 CSV 파일로 저장하시오.
 - 아두이노에 조이스틱과 스위치 5개를 연결한다.
 - 데이터의 입력값 X,Y의 아날로그 데이터, 타겟값은 조이스틱 데이터가 가리키는 방향이다.
 - X, Y축 데이터는 0~1 사이의 실수로 정규화하고, 타겟값은 원핫벡터로 표현한다.
 - 타겟값은 왼쪽, 오른쪽, 중앙, 위, 아래 5개로 구성된다.
ex) [0.965, 0.512, 1, 0, 0, 0, 0]
 - 각 방향별로 200개의 데이터를 수집한다.
- 수집한 데이터를 matplotlib 라이브러리를 사용하여 산점도로 시각화한다.
 - X축과 Y축 데이터를 바탕으로 출력하고, 각 카테고리 별로 서로 다른 색을 이용하여 출력한다.

| 과제 12

모델 학습

- 학습데이터 중 80%는 학습에 사용하며, 20%는 검증에 사용한다.
- MLP 모델을 학습한다.
- 추론모델은 weight.h 헤더 파일로 코드와 함께 업로드

과제 13

모델 추론

- 학습된 모델을 아두이노에 업로드하고 추론 실행
- 1초 간격으로 조이스틱의 아날로그 데이터를 입력
 - 조이스틱 아날로그 데이터를 모델에 맞게 전처리
- 추론을 통하여 조이스틱의 방향 결정
- 결정된 방향을 바탕으로 다음과 같이 동작
 - 브레드보드의 상,하,좌,우,중앙에 각각 LED 1개씩 배치 및 연결
 - 추론한 방향에 맞는 LED를 점등, 그 외의 LED는 소등

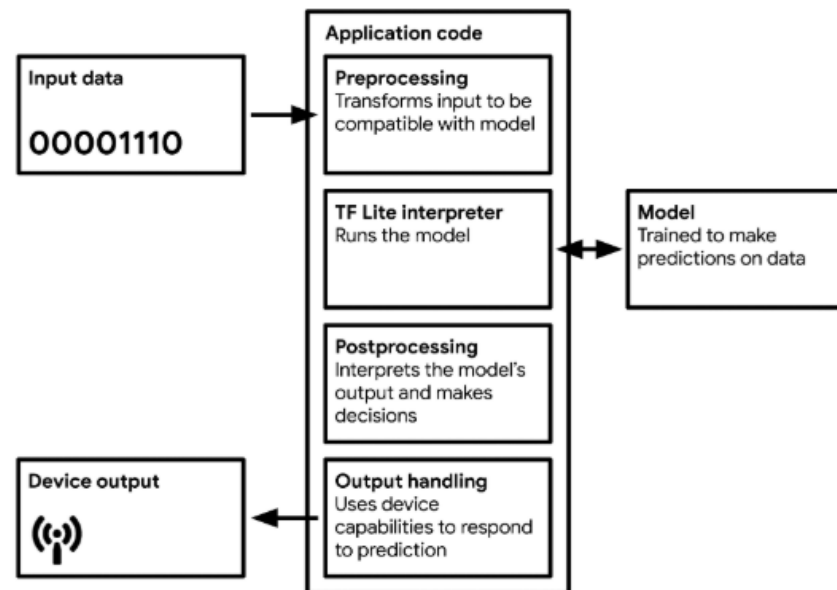


Figure 5-1. A basic TinyML application architecture

| 고생하셨습니다 |

Thank you!